MANUAL TÉCNICO

Sistema IoT de Smart Farming basado en WiFi y MQTT

Documento orientado al cliente para operación básica, diagnóstico y solución de fallas del prototipo.

Proyecto	Prototipo de monitoreo y control agrícola inteligente
Tecnologías base	ESP8266, WiFi, MQTT, sensores y actuadores
Elaborado para	Granja Urkuwayku
Autor base del proyecto	Ricardo Xavier Paredes

Versión 1.0

1. Propósito del manual

Este manual explica de forma técnica y operativa cómo funciona el sistema IoT de Smart Farming, qué componentes lo conforman, cómo se realiza el flujo de datos, cómo operar el sistema a nivel básico y qué acciones ejecutar cuando exista una falla en sensores, conectividad, backend o actuadores.

El documento está pensado para personal cliente con conocimientos básicos o intermedios en operación técnica, de modo que pueda entender la lógica del sistema, verificar estados y resolver incidencias comunes sin depender inmediatamente del desarrollador original.

2. Alcance

- Descripción general de la arquitectura del sistema.
- Inventario funcional de componentes IoT y software asociado.
- Secuencia de operación desde la lectura del sensor hasta la visualización o acción.
- Procedimiento básico de revisión, mantenimiento y troubleshooting.
- Buenas prácticas de uso, energía, conectividad y protección física.

3. Descripción general del sistema

El sistema es un prototipo de Smart Farming de bajo costo basado en dispositivos IoT con conectividad WiFi. Cada nodo usa un módulo ESP8266 para leer sensores o controlar actuadores. La información se transmite hacia un servidor MQTT y puede ser procesada por un backend para almacenamiento, monitoreo y ejecución de reglas de control.

La solución fue concebida para monitorear variables agrícolas y permitir acciones automáticas o manuales sobre el entorno, por ejemplo, ventilación, riego u otras respuestas asociadas a condiciones de temperatura, humedad o concentración de CO_2.

4. Arquitectura funcional

La arquitectura lógica del sistema sigue esta secuencia:

1. Sensores capturan variables del entorno o del cultivo.
2. El ESP8266 procesa la lectura y se conecta a la red WiFi disponible.
3. Los datos se publican en tópicos MQTT hacia el servidor.
4. El backend recibe, procesa y puede almacenar la información.
5. Las reglas del sistema generan decisiones o alertas.
6. Los actuadores reciben comandos y ejecutan acciones físicas.

4.1 Diagramas de conexión de referencia

Como complemento al presente manual, se incorporan los diagramas de conexión utilizados en la tesis para facilitar la instalación, verificación de cableado y mantenimiento de cada nodo IoT.

Estos diagramas deben tomarse como referencia técnica del prototipo. Antes de una instalación definitiva, conviene validar el pinout real del ESP8266, el voltaje del actuador y la lógica HIGH/LOW del relé.

4.1.1 Sensor SCD41

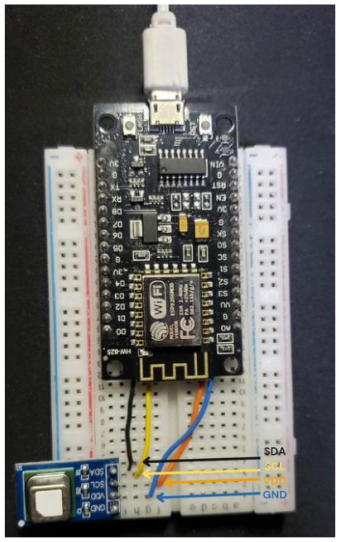

Diagrama de conexión del prototipo.

- VDD -> alimentación del sensor
- GND -> tierra común
- SCL -> D6 / GPIO14 del ESP8266
- SDA -> pin mostrado en el diagrama de tesis

4.1.2 Sensor de humedad del suelo

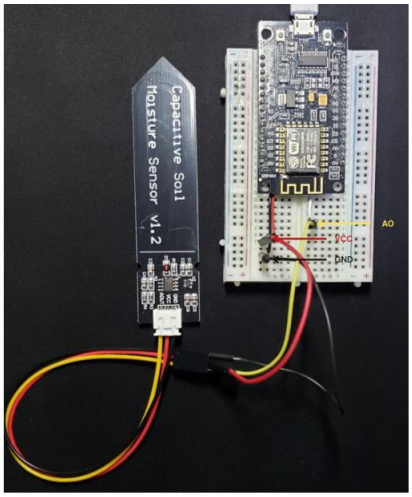

Diagrama de conexión del prototipo.

- VCC -> 3.3 V
- GND -> tierra común
- AOUT -> A0 / ADC0 del ESP8266
- Requiere calibración de seco y húmedo

4.1.3 Sensor DS18B20

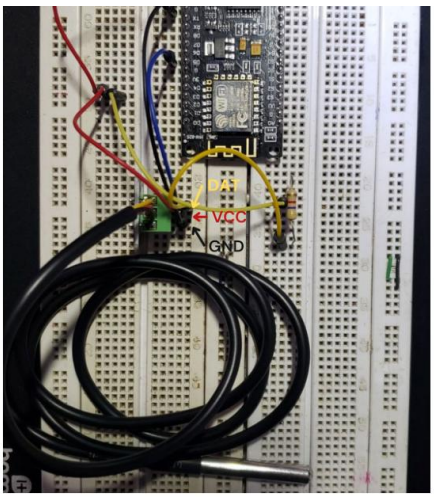

Diagrama de conexión del prototipo.

- VCC -> 3.3 V
- GND -> tierra común
- DAT -> D5 / GPIO14 del ESP8266
- Comunicación 1-Wire

4.1.4 Actuadores por relé

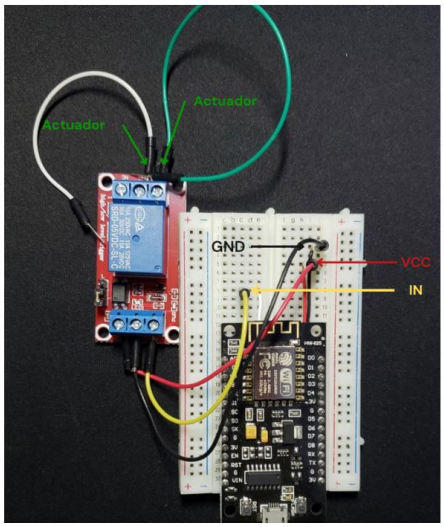

Diagrama de conexión del prototipo.

- VCC -> 3.3 V del relé
- GND -> tierra común
- IN -> D6 / GPIO12 del ESP8266
- El actuador se conecta al relé como interruptor

Nota: para mantenimiento correctivo, compare siempre el cableado físico instalado con estos esquemas y con el firmware cargado en cada módulo.

5. Componentes principales

Componente	Función	Variable / acción	Observaciones	Cant.
ESP8266	Microcontrolador con WiFi integrado	Adquisición, procesamiento y comunicación	Nodo central de cada dispositivo IoT	5
Sensor SCD41	Medición ambiental	CO_2, temperatura y humedad	Sensor digital de precisión	1
Sensor capacitivo HS v1.2	Medición de suelo	Humedad de suelo	Tipo analógico	1
Sensor DS18B20	Medición puntual	Temperatura	Sensor digital cableado	1
Relé electrónico	Interfaz de potencia	Conmutación de actuadores	Aísla control lógico de carga	2

6. Software y servicios utilizados

- Firmware del ESP8266 desarrollado en Arduino IDE.
- Red WiFi local como medio de comunicación de los nodos.
- Servidor MQTT (Mosquitto) como broker de mensajería.
- Backend o aplicación receptora para suscripción, visualización y almacenamiento.

7. Flujo de operación del sistema

7. **Inicialización:** El ESP8266 energiza el nodo, carga la configuración y trata de conectarse a la red WiFi configurada.
8. **Conexión:** Una vez con conectividad IP, el equipo establece sesión con el servidor MQTT.
9. **Lectura:** El sensor asociado entrega la variable medida al ESP8266.
10. **Publicación:** El nodo publica la lectura en el tópico MQTT correspondiente.
11. **Procesamiento:** El backend o la lógica de reglas consume la información recibida.
12. **Acción:** Si la condición configurada se cumple, se emite un comando al actuador mediante un tópico o evento de control.
13. **Retroalimentación:** El sistema registra mensajes, estados y eventos para trazabilidad.

8. Procedimiento normal de uso

- Verificar que exista alimentación eléctrica estable en el nodo IoT y en el router o AP WiFi.
- Comprobar que el dispositivo ESP8266 encienda correctamente.
- Confirmar que la red WiFi configurada esté disponible.
- Revisar que el broker MQTT se encuentre activo.

- Observar en el backend o consola que lleguen mensajes desde el sensor.
- Validar que, ante un evento de control, el relé o actuador responda según lo esperado.

9. Mantenimiento preventivo recomendado

Elemento	Actividad	Frecuencia sugerida	Resultado esperado
Sensores	Limpieza superficial, revisión de cableado y verificación de lecturas atípicas.	Cada 15 a 30 días	Mediciones estables y sin ruido excesivo.
ESP8266	Inspección visual, reinicio controlado y revisión de conectividad.	Mensual	Nodo operativo y con conexión WiFi/MQTT.
Relé o actuador	Prueba de activación y revisión de bornes o cables.	Mensual	Conmutación correcta sin falsos contactos.
Red WiFi	Comprobación de cobertura, SSID, clave y estabilidad del AP/router.	Mensual o ante fallas	Señal suficiente para comunicación continua.
Backend o broker	Validar servicio en ejecución, logs, almacenamiento y puertos.	Semanal	Recepción y registro normal de mensajes.

10. Diagnóstico de fallas y solución

La siguiente matriz resume incidencias comunes en un sistema IoT de este tipo y la acción recomendada para resolverlas.

Síntoma	Causa probable	Verificación	Acción correctiva
El nodo no enciende	Fuente sin energía, cable USB defectuoso o daño físico	Revisar LED, fuente, voltaje y cable	Cambiar cable o fuente, verificar alimentación y reemplazar módulo si no responde.
No hay conexión WiFi	SSID o clave incorrectos, señal baja o AP apagado	Revisar parámetros y disponibilidad de red	Actualizar credenciales, acercar el nodo al AP o reiniciar router.
No conecta al broker MQTT	Broker caído, IP o puerto incorrecto o bloqueo de red	Probar ping, puerto y estado del servicio	Levantar Mosquitto, corregir IP o puerto y revisar firewall.
No llegan datos del sensor	Cableado suelto, sensor dañado o error de firmware	Inspeccionar conexiones y valores crudos	Reconectar, recalibrar o reemplazar sensor; revisar código.

Síntoma	Causa probable	Verificación	Acción correctiva
Lecturas incoherentes	Ruido eléctrico, humedad, mala ubicación o calibración deficiente	Comparar con medición manual o patrón	Aislar cableado, mejorar montaje y recalibrar el sensor.
Actuador no responde	Relé defectuoso, falta de energía en la carga o comando no recibido	Escuchar conmutación, medir salida y revisar tópico	Verificar relé, alimentación de la carga y comando MQTT.
Mensajes intermitentes	Cobertura WiFi débil o reinicios del nodo	Revisar RSSI, logs y estabilidad de energía	Reubicar AP, añadir repetidor y estabilizar alimentación.
Backend no muestra eventos	Suscripción incorrecta, servicio detenido o error de base de datos	Revisar logs y tópicos suscritos	Corregir suscripción, reiniciar servicio y validar almacenamiento.

11. Checklist rápido de troubleshooting

14. Confirmar energía en nodo, router y actuador.
15. Verificar que el ESP8266 obtenga conexión WiFi.
16. Confirmar acceso al servidor MQTT.
17. Revisar que el tópico publicado sea el correcto.
18. Validar cableado y estado físico del sensor o relé.
19. Revisar logs del backend y marcas de tiempo de mensajes.
20. Reiniciar primero nodo, luego router/AP y finalmente broker si es necesario.
21. Sustituir el componente sospechoso cuando la falla ya fue aislada.

12. Buenas prácticas de instalación

- Evitar exposición directa del ESP8266 y conexiones a lluvia, lodo o condensación.
- Usar cajas protectoras y alivio de tensión en los cables.
- Separar líneas de potencia de líneas de señal para reducir interferencia.
- Rotular cada nodo, sensor y actuador para facilitar mantenimiento.
- Respaldar el firmware y la configuración de red y MQTT antes de cambios importantes.

13. Parámetros técnicos de referencia del prototipo

Tecnología de comunicación principal	WiFi con integración a servidor MQTT
Cobertura máxima observada	33,8 m

Área de cobertura estimada	3589,08 m^2
RSSI promedio reportado	-65,44 dBm
Consumo energético del sistema evaluado	10,21 Wh en 5 horas (0,01021 kWh)

14. Limitaciones conocidas del prototipo

- El sistema corresponde a un prototipo funcional y puede requerir encapsulado industrial para operación prolongada en campo.
- La cobertura depende de obstáculos, distribución física y ubicación del punto de acceso WiFi.
- La confiabilidad final del control automático depende de la correcta calibración de sensores y de la lógica implementada en backend o reglas.
- La electroválvula no figura como validada experimentalmente en el resumen del proyecto, por lo que cualquier integración de riego debe probarse antes de operación continua.

15. Recomendaciones de mejora

- Añadir protección física para uso exterior.
- Optimizar el consumo energético y evaluar baterías o energía solar.
- Mejorar cobertura WiFi mediante reubicación del router o repetidores.
- Escalar el sistema con más sensores y actuadores según la necesidad del cultivo.

16. Anexo: datos a personalizar antes de la entrega final

Antes de entregar este manual al cliente final, conviene completar los siguientes datos reales de la implementación:

- SSID de la red WiFi y procedimiento de cambio de credenciales.
- IP o dominio del broker MQTT y puertos utilizados.
- Lista exacta de tópicos MQTT por sensor y actuador.
- Versión de firmware cargada en cada ESP8266.
- Ubicación física de cada nodo en la finca o invernadero.
- Responsable de soporte y contacto técnico.

17. Firmware implementado y estructura de códigos

Los códigos fuente desarrollados para el prototipo fueron implementados en Arduino IDE 2.3.2 utilizando el microcontrolador ESP8266 como nodo principal de comunicación WiFi y MQTT.

Cada archivo .ino cumple una función específica dentro del sistema: sensado ambiental, sensado de suelo o control de actuadores. En conjunto, estos programas materializan la lógica operativa del prototipo descrito en este manual.

Por seguridad, en los anexos se mantienen enmascaradas las credenciales WiFi. Antes de una puesta en producción, se recomienda parametrizar SSID, contraseña, dirección IP del broker MQTT, puertos y tópicos en una hoja de configuración controlada.

Archivo	Función	Tópicos MQTT	Modo
MQTT-SCD41-ESP8266.ino	Lectura de temperatura, humedad y CO2 del ambiente con SCD41 y publicación MQTT.	temperaturaH, humedadH, CO2H	Publicador
SoilMoisture-MQTT.ino	Lectura analógica del sensor capacitivo de humedad del suelo y envío del porcentaje.	SoilMoistureW	Publicador
DS18B20-ESP8266-MQTT.ino	Lectura de temperatura del suelo mediante DS18B20 y publicación MQTT.	TemperaturaTW	Publicador
BombaDeAgua-MQTT-TEST-2Temas.ino	Recepción de comandos MQTT para activar o desactivar el extractor mediante relé.	activarExtractor / desactivarExtractor	Suscriptor
Electrovalvula-MQTT-TEST-2Temas.ino	Recepción de comandos MQTT para activar o desactivar la electroválvula mediante relé.	activarValvula / desactivarValvula	Suscriptor

18. Recomendaciones técnicas derivadas del firmware

- Usar clientId MQTT único por dispositivo para evitar desconexiones cuando varios nodos se conecten al mismo broker.
- Estandarizar los nombres de tópicos MQTT y documentarlos por variable, nodo y actuador.
- Agregar publicación de estado del nodo (online/offline), marca de tiempo y último valor válido.
- Implementar validaciones de rango para sensores, especialmente humedad del suelo, temperatura y CO2.
- Sustituir direcciones IP fijas por parámetros configurables o nombres DNS cuando el sistema escale.
- Definir claramente la lógica HIGH/LOW de los relés, porque puede variar según el módulo utilizado.
- Evaluar la incorporación de watchdog, reintentos no bloqueantes y almacenamiento temporal de datos ante pérdida de conectividad.

19. Anexos de código fuente

A continuación, se incorporan los códigos desarrollados para el prototipo, con una breve explicación funcional para facilitar la operación, mantenimiento y futuras modificaciones del sistema.

19.1 MQTT-SCD41-ESP8266.ino

Nodo de monitoreo ambiental. Inicializa el bus I2C del SCD41, obtiene mediciones de CO2, temperatura y humedad relativa, y publica cada variable en un tópico MQTT independiente cada 2 minutos.

Código fuente:

```
#include <Wire.h>
#include "SparkFun_SCD4x_Arduino_Library.h"
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// Pinout for ESP8266
#define SDA_PIN 14 //D5 - GPIO14
```

```
#define SCL_PIN 12 //D6 - GPIO12

SCD4x sensor;

float temp;
float humidity;
float co2;



// Variables de conexión WiFi
const char* ssid = "************";
const char* password = "***********";

// Variables de conexión MQTT
const char* mqtt_server = "192.168.0.24"; // Dirección IP del servidor MQTT
const char* mqtt_topic_temp = "temperaturaH";
const char* mqtt_topic_hume = "humedadH";
const char* mqtt_topic_co2 = "CO2H";

WiFiClient espClient;
PubSubClient client(espClient);



void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("Dirección IP:");
  Serial.println(WiFi.localIP());
}



void reconnect() {
  // Loop hasta que estemos reconectados
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    String clientId = "ESP8266Client-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println("Conectado");
    } else {
      // Depurar: Imprimir el estado de la conexión MQTT
      Serial.print("Falló, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando de nuevo en 5 segundos");
      delay(5000);
    }
  }
}



void setup() {
```

```
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN);
  if (!sensor.begin()) {
    Serial.println("Error al iniciar el sensor!");
    while (1);
  }
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  if (sensor.readMeasurement()) {
    co2 = sensor.getCO2();
    temp = sensor.getTemperature();
    humidity = sensor.getHumidity();

    Serial.print("CO2: ");
    Serial.print(co2);
    Serial.print(" ppm, Temperatura: ");
    Serial.print(temp);
    Serial.print(" °C, Humedad: ");
    Serial.print(humidity);
    Serial.println(" %");

    // Publicar datos en los temas MQTT correspondientes
    String co2String = String(co2);
    String tempString = String(temp);
    String humString = String(humidity);

    // Publicar CO2
    String co2Payload = "{" + co2String + "}";
    Serial.print("Publicando mensaje de CO2: ");
    Serial.println(co2Payload);
    client.publish(mqtt_topic_co2, co2Payload.c_str());

    // Publicar temperatura
    String tempPayload = "{" + tempString + "}";
    Serial.print("Publicando mensaje de temperatura: ");
    Serial.println(tempPayload);
    client.publish(mqtt_topic_temp, tempPayload.c_str());

    // Publicar humedad
    String humPayload = "{" + humString + "}";
    Serial.print("Publicando mensaje de humedad: ");
    Serial.println(humPayload);
    client.publish(mqtt_topic_hume, humPayload.c_str());

    // Measure and print WiFi signal strength (RSSI)
    long rssi = WiFi.RSSI();
    Serial.print("Intensidad de señal Wi-Fi (RSSI): ");
    Serial.print(rssi);
    Serial.println(" dBm");



  } else {
    Serial.println("Error al leer el sensor!");
  }

  delay(120000); //2 minutos
```

```
}
```

19.2 SoilMoisture-MQTT.ino

Nodo de humedad del suelo. Lee el valor analógico del sensor capacitivo, convierte la lectura a porcentaje usando valores de calibración seco/húmedo y la publica en el broker MQTT.

Código fuente:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>



// Definir los pines del sensor
const int sensorPin = A0;

//Valores de seco y húmedo
const int dry = 700; // value for dry sensor
const int wet = 300; // value for wet sensor

// Variables de conexión WiFi
const char* ssid = "***********";
const char* password = "***********";

// Variables de conexión MQTT
const char* mqtt_server = "192.168.0.18"; // Dirección IP del servidor MQTT
const char* mqtt_topic_soil_moisture = "SoilMoistureW";



WiFiClient espClient;
PubSubClient client(espClient);



void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("Dirección IP:");
  Serial.println(WiFi.localIP());
}



void reconnect() {
  // Loop hasta que estemos reconectados
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    String clientId = "ESP8266Client-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println("Conectado");
```

```
    } else {
      // Depurar: Imprimir el estado de la conexión MQTT
      Serial.print("Falló, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando de nuevo en 5 segundos");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  // Leer el valor analógico del sensor
  int sensorValue = analogRead(sensorPin);
  // Convertir el valor a porcentaje de humedad
  int percentage = map(sensorValue, wet, dry, 0, 100);

  // Crear un mensaje para enviar al servidor MQTT
  String message = String(percentage);

  // Publicar el mensaje en el tópico MQTT
  client.publish(mqtt_topic_soil_moisture, message.c_str());

  // Measure and print WiFi signal strength (RSSI)
  long rssi = WiFi.RSSI();
  Serial.print("Intensidad de señal Wi-Fi (RSSI): ");
  Serial.print(rssi);
  Serial.println(" dBm");

  delay(120000); // Esperar 2 minutos antes de la siguiente lectura
}
```

19.3 DS18B20-ESP8266-MQTT.ino

Nodo de temperatura del suelo. Usa comunicación OneWire para leer el DS18B20, publica la temperatura por MQTT y además imprime el RSSI para validar calidad del enlace WiFi.

Código fuente:

```
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// Define pin where the DS18B20 is connected
#define oneWireBus 14 // Pin D5

// Setup a oneWire instance to communicate with any OneWire devices
OneWire oneWire(oneWireBus);

// Pass oneWire reference to DallasTemperature library
DallasTemperature sensors(&oneWire);
```

```
// Variables de conexión WiFi
const char* ssid = "***********";
const char* password = "***********";

// Variables de conexión MQTT
const char* mqtt_server = "192.168.0.18"; // Dirección IP del servidor MQTT
const char* mqtt_topic_temp = "TemperaturaTW";

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("Dirección IP:");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  // Loop hasta que estemos reconectados
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    String clientId = "ESP8266Client-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println("Conectado");
    } else {
      // Depurar: Imprimir el estado de la conexión MQTT
      Serial.print("Falló, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando de nuevo en 5 segundos");
      delay(5000);
    }
  }
}

void setup() {
  // Start serial communication for debugging
  Serial.begin(115200);
  // Start the DS18B20 sensor
  sensors.begin();
  // Connect to WiFi
  setup_wifi();
  // Set MQTT server
  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
```

```
  // Request temperature readings from the sensor
  sensors.requestTemperatures();
  // Fetch the temperature in Celsius
  float temperatureC = sensors.getTempCByIndex(0);
  // Print the temperature to the Serial Monitor
  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println("ºC");

  // Create a message to send to the MQTT server
  String message = String(temperatureC);

  // Publish the message to the MQTT topic
  client.publish(mqtt_topic_temp, message.c_str());

  // Measure and print WiFi signal strength (RSSI)
  long rssi = WiFi.RSSI();
  Serial.print("Intensidad de señal Wi-Fi (RSSI): ");
  Serial.print(rssi);
  Serial.println(" dBm");

  // Add a delay between readings
  delay(120000);
}
```

19.4 BombaDeAgua-MQTT-TEST-2Temas.ino

Nodo actuador por relé. Se suscribe a dos tópicos para activación y desactivación del extractor de aire; al recibir los comandos esperados conmuta la salida digital conectada al relé.

Código fuente:

```
// Librerías
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// Configuración Wi-Fi
const char* ssid = "************";
const char* password = "************";

// Configuración MQTT
const char* mqtt_server = "DIRECCION_DEL_SERVIDOR_MQTT"; //Dirección del MQTT
const int mqtt_port = 1883;
const char* topic_activar = "activarExtractor";
const char* topic_desactivar = "desactivarExtractor";

// Pin del relé
#define RELAY_PIN 12

WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);

  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
}
```

```
void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
  String message;

  for (int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("Mensaje recibido en tópico [");
  Serial.print(topic);
  Serial.print("]: ");
  Serial.println(message);

  if (String(topic) == topic_activar) {
    if (message == "ACTIVAR") {
      activarExtractor();
    }
  } else if (String(topic) == topic_desactivar) {
    if (message == "DESACTIVAR") {
      desactivarExtractor();
    }
  }
}

void activarExtractor() {
  digitalWrite(RELAY_PIN, HIGH);
  Serial.println("Extractor de aire activado");
}

void desactivarExtractor() {
  digitalWrite(RELAY_PIN, LOW);
  Serial.println("Extractor de aire desactivado");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    if (client.connect("ESP8266Client")) {
      Serial.println("Conectado");
      client.subscribe(topic_activar);
      client.subscribe(topic_desactivar);
    } else {
      Serial.print("Falló, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando de nuevo en 5 segundos");
```

```
    delay(5000);
   }
  }
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
}
```

19.5 Electrovalvula-MQTT-TEST-2Temas.ino

Nodo actuador por relé para riego. Se suscribe a los tópicos de activación y desactivación de la electroválvula y ejecuta el cambio de estado del relé cuando recibe el mensaje MQTT correspondiente.

Código fuente:

```
// Librerías
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// Configuración Wi-Fi
const char* ssid = "***********";
const char* password = "***********";

// Configuración MQTT
const char* mqtt_server = "192.168.0.24"; //Dirección del MQTT
const int mqtt_port = 1883;
const char* topic_activar = "activarValvula";
const char* topic_desactivar = "desactivarValvula";

// Pin del relé
#define RELAY_PIN 12

WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);

  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
}

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

```
  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
  String message;

  for (int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("Mensaje recibido en tópico [");
  Serial.print(topic);
  Serial.print("]: ");
  Serial.println(message);

  if (String(topic) == topic_activar) {
    if (message == "ACTIVAR") {
      activarValvula();
    }
  } else if (String(topic) == topic_desactivar) {
    if (message == "DESACTIVAR") {
      desactivarValvula();
    }
  }
}

void activarValvula() {
  digitalWrite(RELAY_PIN, HIGH);
  Serial.println("Electroválvula activada");
}

void desactivarValvula() {
  digitalWrite(RELAY_PIN, LOW);
  Serial.println("Electroválvula desactivada");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    if (client.connect("ESP8266Client")) {
      Serial.println("Conectado");
      client.subscribe(topic_activar);
      client.subscribe(topic_desactivar);
    } else {
      Serial.print("Falló, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando de nuevo en 5 segundos");
      delay(5000);
    }
  }
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
}
```